

Sanskrit Document Retrieval-Augmented Generation (RAG) System

1. Introduction

The rapid growth of Generative AI has made it possible to build systems that can understand natural language and generate human-like responses. However, a general-purpose language model does not always have access to specific information contained in a private or domain-specific document. It may also generate information that is not present in the source data.

To solve this problem, I developed a **Retrieval-Augmented Generation (RAG) system for Sanskrit documents**.

The main purpose of this project is to create a question-answering system that can take a Sanskrit document as its knowledge source, retrieve the most relevant information from that document, and then use a language model to generate an answer based on the retrieved context.

The system combines two important capabilities:

1. **Retrieval** – finding the most relevant information from the Sanskrit document.
2. **Generation** – using a language model to convert the retrieved information into a readable answer.

The project was specifically designed to work in a **CPU-only environment**, without depending on paid cloud APIs. This makes the system lightweight, locally executable, and suitable for environments where GPU resources are not available.

The final system provides a simple Streamlit interface where a user can enter a question in Sanskrit or transliterated/mixed text and receive an answer along with the source pages used by the system.

2. Problem Statement

Traditional document search systems generally depend on exact keyword matching. This can become difficult when the user asks a question using different words, sentence structures, or transliteration.

For example, the information may exist in a document, but the exact words used in the user's question may not appear in the same form in the document.

A second problem is that a normal language model may generate an answer based on its general training knowledge instead of the specific document provided by the user.

Therefore, the problem addressed by this project is:

To develop a lightweight, CPU-based Sanskrit question-answering system that can retrieve relevant information from Sanskrit documents and generate meaningful answers using a local language model.

The system should be able to:

- Process Sanskrit PDF documents.
- Clean and prepare the extracted text.

- Divide the document into meaningful chunks.
 - Convert document chunks into semantic vector representations.
 - Retrieve relevant chunks for a user query.
 - Generate an answer using the retrieved information.
 - Display source pages for transparency.
 - Work without requiring a GPU or paid API.
-

3. Objective of the Project

The main objective of this project is to demonstrate an end-to-end **Retrieval-Augmented Generation pipeline for Sanskrit documents**.

The specific objectives are:

- To load Sanskrit documents from PDF format.
 - To extract text page by page.
 - To preprocess and clean the extracted text.
 - To divide the document into smaller overlapping chunks.
 - To generate multilingual semantic embeddings.
 - To store the embeddings in a local vector database/index.
 - To perform semantic similarity-based retrieval.
 - To retrieve the top relevant document chunks for a query.
 - To pass retrieved context to a local language model.
 - To generate an answer based on the retrieved context.
 - To provide a simple user interface using Streamlit.
 - To evaluate retrieval quality, semantic similarity, and response latency.
 - To maintain a modular architecture where retrieval and generation can be independently improved.
-

4. Purpose of Using RAG

The main purpose of RAG in this project is to **connect the language model with the actual information contained in the Sanskrit document**.

A normal LLM works mainly from the knowledge it learned during training. It does not automatically know the contents of a new PDF supplied by the user.

RAG solves this by following the process:

User Question → Search Document → Retrieve Relevant Information → Give Information to LLM → Generate Answer

This provides two major advantages.

4.1 Document-Based Answers

Instead of depending completely on the model's internal knowledge, the system provides the model with information retrieved from the actual document.

4.2 Source Transparency

The system also displays the retrieved page numbers and similarity scores. This allows the user to understand which parts of the document were considered relevant.

Therefore, the purpose of RAG is not simply to generate text. Its purpose is to make the generation process **more grounded in the provided document**.

5. Project Workflow

The complete workflow of the system is divided into two main phases:

Phase 1: Document Indexing

The document is prepared before the user asks a question.

Sanskrit PDF

↓

PDF Text Extraction

↓

Text Cleaning

↓

Text Chunking

↓

Embedding Generation

↓

FAISS Vector Index

↓

Stored Local Knowledge Base

Phase 2: Query Processing

When the user asks a question:

User Question



Question Embedding



FAISS Similarity Search



Top-K Relevant Chunks



Context Formation



Qwen Language Model



Generated Answer



Answer + Source Pages

6. System Architecture

The architecture of the project follows a modular RAG design.

Sanskrit PDF



Document Loader



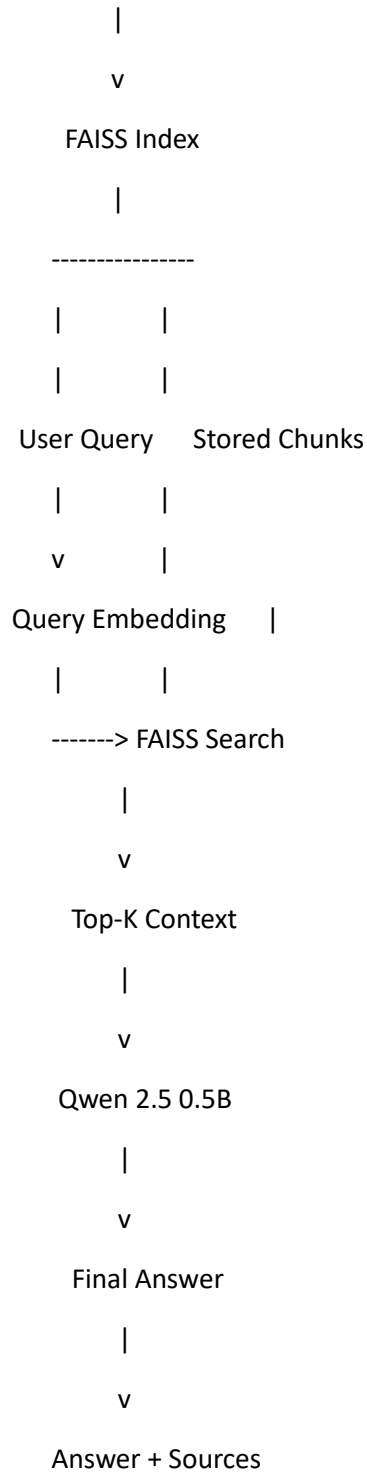
Preprocessing



Chunk Generation



Multilingual Embedding Model



The architecture is modular because each major task is handled by a separate Python file.

7. Methodology

The methodology followed in the project consists of the following steps.

Step 1: Document Loading

The Sanskrit PDF is loaded page by page using the pypdf library.

Each page is stored with:

- Page number
- Extracted text

The page number is retained as metadata so that retrieved information can later be traced back to the original document.

During implementation, the system successfully loaded:

11 pages

8. Text Preprocessing

Raw PDF text is not always directly suitable for NLP processing.

PDF extraction may introduce:

- Unnecessary line breaks
- Extra spaces
- Tabs
- Formatting inconsistencies

Therefore, basic preprocessing is performed before generating embeddings.

The preprocessing step:

- Replaces line breaks with spaces.
- Removes unnecessary whitespace.
- Normalizes the extracted text.
- Keeps the original Sanskrit content as much as possible.

This approach was intentionally kept simple because aggressive modification of Sanskrit text could accidentally change its meaning.

9. Text Chunking

After preprocessing, the document is divided into smaller pieces called **chunks**.

The current implementation uses approximately:

- **Chunk size:** 500 characters
- **Overlap:** 100 characters

The overlap is important because a sentence or meaningful piece of information may fall across the boundary of two chunks.

For example:

Chunk 1:

... information A B C D E ...

Chunk 2:

... D E F G H ...

Here, D and E are repeated between chunks.

This helps prevent important information from being lost during retrieval.

The implemented system generated:

26 chunks from 11 pages.

10. Embedding Generation

After chunking, each chunk is converted into a numerical vector called an **embedding**.

The embedding model used is:

sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

The model converts text into a **384-dimensional vector**.

For example:

Sanskrit text



Embedding Model



[0.053, -0.041, -0.001, ...]

The numerical representation captures semantic information about the text.

Therefore, two pieces of text with similar meaning can have similar vector representations even when their exact wording is different.

11. Why I Used a Multilingual Embedding Model

I selected paraphrase-multilingual-MiniLM-L12-v2 because the project involves Sanskrit and may also receive transliterated or mixed-language queries.

Using an English-only embedding model would not be an ideal choice for this use case.

The multilingual model provides a better foundation for:

- Sanskrit text
- Multilingual queries

- Transliteration/mixed-language input
- Semantic similarity

Another important reason is that the model is relatively lightweight compared with much larger transformer models, making it more practical for CPU-based execution.

12. Vector Database / FAISS

Once embeddings are generated, they need to be stored so that the system can efficiently search them.

For this project, I used:

FAISS

FAISS stands for **Facebook AI Similarity Search**.

It is designed for efficient similarity search over vectors.

The project uses:

IndexFlatIP

where IP means **Inner Product**.

Because the embeddings are normalized, inner-product similarity behaves similarly to cosine similarity.

The system stores:

- Chunk embedding
- Chunk text
- Page number

When a user enters a query, the query is also converted into an embedding and compared with the stored vectors.

The system then returns the most similar chunks.

13. Why I Used FAISS

FAISS was selected for several reasons.

1. Lightweight

The current dataset contains only 26 chunks, so a large external vector database is unnecessary.

2. Local

FAISS works locally and does not require a cloud service.

3. Fast Retrieval

It is specifically designed for vector similarity search.

4. Easy Integration

FAISS integrates naturally with Python and NumPy.

5. Suitable for the Assignment

The assignment requires a lightweight and locally runnable implementation, making FAISS a suitable choice.

For a much larger dataset, the project can later move to an approximate FAISS index or a dedicated vector database.

14. Retrieval Process

The retrieval process works as follows:

1. User enters a question.
2. The question is converted into an embedding.
3. FAISS compares the question vector with document vectors.
4. Similarity scores are calculated.
5. The top 3 relevant chunks are selected.
6. These chunks are passed to the generation model.

The project currently uses:

Top-K = 3

This means the system retrieves the three most relevant chunks.

For example, one of the tests returned:

Retrieved Pages: [6, 4, 5]

This demonstrates that the retrieval component is actively searching the indexed document rather than directly generating an answer without document retrieval.

15. Generation Model

The generation component uses:

Qwen/Qwen2.5-0.5B-Instruct

This is a relatively small instruction-tuned language model.

The model receives:

- User question
- Retrieved document context

The prompt follows the general structure:

Context:

[Retrieved document information]

Question:

[User question]

Answer:

The model then generates the final response.

16. Why I Used Qwen 2.5 0.5B

The main reason for selecting Qwen 2.5 0.5B was the **CPU-only requirement**.

Large language models can require significant GPU memory and computational resources.

A 0.5B parameter model provides a practical balance between:

- Local execution
- Lower memory requirements
- CPU compatibility
- Instruction following
- Generative capabilities

The model also avoids the need for a paid external API.

However, there is a clear trade-off.

A smaller model is generally less capable than larger models in:

- Complex reasoning
- Multilingual understanding
- Sanskrit understanding
- Following detailed instructions
- Producing highly accurate explanations

This limitation was visible during evaluation and is discussed later in this report.

17. Prompting Strategy

The generation model is provided with the retrieved context and the user question.

The basic idea is:

Use the provided context to answer the question.

This helps reduce the possibility of the model answering entirely from its general knowledge.

The retrieved information therefore acts as the knowledge source for generation.

A future improvement would be to make the prompt more restrictive, for example:

Answer only from the provided context.

If the answer is not present in the context, say:

"Information not found in the provided document."

This would improve grounding and reduce hallucination.

18. Technology Stack

Technology	Purpose
Python	Main programming language
pdfplumber	PDF text extraction
Regular Expressions	Text cleaning
Sentence Transformers	Text embeddings
MiniLM	Multilingual semantic representation
NumPy	Numerical vector processing
FAISS	Vector similarity search
Pickle	Local metadata storage
PyTorch	CPU-based model execution
Hugging Face Transformers	Loading Qwen model
Qwen 2.5 0.5B	Answer generation
Streamlit	User interface
Git/GitHub	Version control and project sharing

19. Libraries Used and Why

Python

Python was selected because it has a strong ecosystem for:

- Machine Learning
- NLP
- Deep Learning
- Vector databases
- Data processing
- Web applications

It also allows all components of the RAG system to be integrated easily.

pypdf

Used for extracting text from the Sanskrit PDF.

It is:

- Lightweight
 - Open-source
 - Easy to use
 - Suitable for PDF-based document ingestion
-

Regular Expressions (re)

Used for simple preprocessing operations such as:

- Removing extra whitespace
- Replacing line breaks
- Cleaning formatting artifacts

It avoids introducing unnecessary complexity into the preprocessing pipeline.

Sentence Transformers

Used to convert document chunks and user questions into embeddings.

This is important because FAISS works with numerical vectors rather than raw text.

NumPy

Used to:

- Store vectors
- Convert embeddings into arrays

- Convert vectors into float32
 - Prepare data for FAISS
-

FAISS

Used for semantic vector retrieval.

It allows the system to find the chunks that are mathematically closest to the query vector.

Pickle

Used to save the document chunk metadata and page information alongside the FAISS index.

This allows the index to be reused later without rebuilding the entire dataset.

PyTorch

Used as the deep learning framework for running the Qwen language model.

The project explicitly uses CPU execution.

Transformers

Used to:

- Load the tokenizer
 - Load the Qwen model
 - Tokenize prompts
 - Generate responses
-

Streamlit

Used to create the front-end interface.

Instead of building a separate frontend using React or another framework, Streamlit provides a simple way to demonstrate the RAG system.

20. Why I Used Semantic Search Instead of Keyword Search

Keyword search looks for exact words.

For example:

Question:

Who was Shankhanada's servant?

A keyword system may search for exact terms such as:

Shankhanada

servant

However, semantic retrieval converts the text into vectors and compares meaning.

This is more suitable when:

- Words are written differently.
- The question structure changes.
- Transliteration is used.
- The user does not exactly copy the wording from the document.

Therefore, semantic retrieval was preferred for this project.

21. Why I Used a RAG Architecture Instead of Only an LLM

Using only an LLM would not guarantee that the answer comes from the supplied Sanskrit document.

The RAG architecture provides a connection between:

Document Knowledge + Language Model

The retriever finds the relevant information, while the generator converts it into a natural-language response.

This separation also makes the system easier to improve.

For example:

- Better embeddings → improved retrieval
 - Better FAISS configuration → improved search
 - Better LLM → improved answer generation
 - Better prompt → improved grounding
-

22. Code Architecture

The project was divided into separate modules so that every component has a clear responsibility.

config.py

Contains:

- File paths
- Model names

- Vector database path
- Top-K value

document_loader.py

Responsible for:

- Loading PDF
- Extracting text
- Maintaining page information

preprocessing.py

Responsible for:

- Cleaning text
- Creating chunks

embeddings.py

Responsible for:

- Loading embedding model
- Creating embeddings

vector_store.py

Responsible for:

- Creating FAISS index
- Saving index
- Loading index
- Searching vectors

build_index.py

Connects:

PDF

→ preprocessing

→ chunks

→ embeddings

→ FAISS

retriever.py

Responsible for:

Question

- embedding
- vector search
- top-k results

generator.py

Responsible for:

Question + Context

- Qwen
- Answer

rag_pipeline.py

Combines retrieval and generation.

app.py

Provides the Streamlit user interface.

evaluation.py

Tests the complete pipeline and measures:

- Generated answer
- Expected answer
- Similarity
- Latency
- Retrieved pages

23. Evaluation Methodology

To evaluate the system, I created test questions with expected answers.

For every question, the evaluation process records:

1. Question
2. Expected answer
3. Generated answer
4. Semantic similarity
5. Latency
6. Retrieved pages

This allows the evaluation to examine not only whether the system produces an answer, but also whether the answer is semantically close to the expected answer and how long the system takes.

24. Evaluation Results

The current evaluation produced the following results from the implemented test cases.

Test Case 1

Question:

शंखनादः कस्य भृत्यः आसीत्?

Expected:

शंखनादः गोवर्धनदासस्य भृत्यः आसीत्।

Generated answer:

The model generated an English explanation about the deity being called Shakhanada and discussed a verse, rather than directly answering the expected question.

Similarity:

2.600

Latency:

36.49 seconds

Retrieved Pages:

[6, 4, 5]

Test Case 2

Question:

शंखनादः कुतः गच्छति?

Expected:

शंखनादः आपणं गच्छति।

Generated answer:

The model interpreted the question incorrectly and generated an English explanation about a king's wife/queen instead of providing the expected answer.

Similarity:

2.248

Latency:

40.66 seconds

Retrieved Pages:

[1, 6, 1]

Test Case 3

Question:

गोवर्धनदासः शंखनादं किम् आदिशत्?

Expected:

गोवर्धनदासः शंखनादं आपणं गत्वा गुरुकं ...

Generated response:

The model stated that the provided context did not contain sufficient information and instead discussed King Bhoj and a reward for reading poems.

Similarity:

1.855

The screenshot did not show the complete latency and retrieved-page information for this test, so those values are not assumed.

25. Important Explanation of the Accuracy

The values:

- 2.600
- 2.248
- 1.855

should **not be reported as accuracy percentages**.

They are semantic similarity scores produced by the current evaluation implementation.

Therefore, it would be technically incorrect to write:

"The model has 86% accuracy."

or

"The model achieved 2.6% accuracy."

The current evaluation does not provide that type of accuracy.

Instead, these values should be described as:

Preliminary semantic similarity scores between the generated answer and the expected answer.

For a stronger final evaluation, I would normalize both embeddings and calculate cosine similarity. That would produce an easier-to-understand score, generally between **0 and 1**, where higher similarity indicates greater semantic closeness.

26. Model Accuracy

The current evaluation shows that the **retrieval component is functioning**, but the final answer generation is not yet consistently accurate for Sanskrit questions.

This is an important distinction.

The system has successfully:

- Loaded the document.
- Created chunks.
- Generated embeddings.
- Built the FAISS index.
- Retrieved document chunks.
- Returned source pages.
- Passed context to the language model.
- Generated responses.

Therefore, the core RAG pipeline is successfully implemented.

The main improvement area is the **generation stage**.

The 0.5B language model sometimes misunderstands Sanskrit questions or generates an English interpretation instead of giving a direct answer.

This is expected to some extent because the selected model was primarily chosen for its lightweight CPU-friendly nature.

27. Faithfulness Evaluation

Faithfulness measures whether the generated answer is actually supported by the retrieved context.

This is particularly important for RAG systems.

A response can be grammatically correct but still be unfaithful if it introduces information that does not exist in the retrieved document.

Test 1

The expected answer is related to Shankhanada being Govardhanadasa's servant.

However, the generated response discussed the deity Shakhana and information that did not directly answer the question.

Therefore:

Faithfulness: Low for this test.

Test 2

The expected answer is:

शंखनादः आपणं गच्छति।

However, the model generated an unrelated English interpretation about a king's wife.

Therefore:

Faithfulness: Low for this test.

Overall Faithfulness

The current generation stage shows **limited faithfulness on the first two visible evaluation cases**.

However, this does not mean the entire RAG architecture is ineffective.

It indicates that the retrieved information is not always being correctly interpreted by the small generation model.

This gives a clear direction for improvement:

Improve retrieval quality + improve prompt grounding + use a stronger/quantized multilingual generation model.

28. Retrieval Performance

The retrieval component is one of the positive aspects of the project.

The system successfully retrieves source pages for user questions.

For example:

Retrieved Pages: [6, 4, 5]

and:

Retrieved Pages: [1, 6, 1]

The Streamlit application also displays similarity scores for retrieved results.

This confirms that the vector retrieval mechanism is operational.

The current problem is not that the system completely fails to retrieve data. Instead, the retrieved information is not always converted into the correct final answer.

29. Latency Evaluation

The first two visible tests produced:

Test Latency

Test 1 36.49 seconds

Test 2 40.66 seconds

The average latency for these two tests is approximately:

38.58 seconds

Is this latency good or bad?

For a real-time interactive question-answering application, **36–41 seconds is relatively high**.

It would feel slow to a user who expects an immediate answer.

However, the latency is understandable because the entire system is running on **CPU**, including:

- Query embedding
- Vector retrieval
- Local language model inference

The FAISS search itself is very small because only 26 chunks are currently stored.

Therefore, the major latency is likely coming from the **CPU-based language model generation**, rather than vector search.

This is an important observation from the performance evaluation.

30. Why CPU Latency Is High

The generation model still needs to perform transformer inference for every question.

Even though Qwen 2.5 0.5B is a small model compared with large LLMs, CPU inference can still be slow.

The current system uses:

CPU

↓

Tokenizer

↓

Qwen 0.5B

↓

Token generation

The model generates tokens sequentially, which increases response time.

Therefore, the current latency should be described as:

Acceptable for a proof-of-concept CPU-only internship assignment, but high for a production interactive application.

This is a positive but technically honest interpretation.

31. Computational Complexity

Let:

- N = number of document chunks
- D = embedding dimension
- T = total document text
- K = number of retrieved chunks

In this project:

$N = 26$

$D = 384$

$K = 3$

Text preprocessing

The text is scanned to clean and normalize it.

Approximate complexity:

$O(T)$

Chunking

The document is divided into chunks.

Approximate complexity:

$O(T)$

Vector Search

FAISS uses IndexFlatIP.

For exact similarity search, the approximate complexity per query is:

$O(N \times D)$

With only:

26×384

vectors and dimensions, the retrieval workload is very small.

Storage Complexity

Embedding storage is approximately:

$O(N \times D)$

LLM Generation

The generation complexity depends on:

- Model size
- Input length
- Number of generated tokens

This is currently the most computationally expensive part of the pipeline.

Therefore, the evaluation suggests that:

Vector retrieval is computationally lightweight, while CPU-based LLM generation is the primary performance bottleneck.

32. Strengths of the Project

The project has several important strengths.

1. Complete End-to-End RAG Pipeline

The project covers the complete workflow from document ingestion to answer generation.

2. CPU-Based

The system satisfies the CPU-only requirement.

3. Local Execution

No paid external API is required.

4. Modular Design

The retriever and generator are implemented separately, making future improvements easier.

5. Multilingual Embeddings

The embedding model is suitable for multilingual data and provides a better foundation for Sanskrit queries.

6. Source Transparency

Retrieved page numbers and similarity scores are displayed to the user.

7. Simple User Interface

Streamlit provides an easy way to demonstrate the system.

8. Reusable Vector Index

The document embeddings are stored locally and do not need to be regenerated for every query.

33. Limitations

The current system has five main limitations.

1. Sanskrit PDF Extraction Quality

The extracted PDF text contains some formatting and character-level artifacts. This can affect embedding and retrieval quality.

2. Small Generation Model

The Qwen 0.5B model is lightweight and CPU-friendly, but its Sanskrit understanding and reasoning capability are limited compared with larger models.

3. High CPU Latency

The first two evaluation cases took 36.49 and 40.66 seconds, which is relatively high for an interactive application.

4. Limited Evaluation Dataset

The current evaluation uses a small manually prepared test set, so it is not sufficient to make broad claims about overall model accuracy.

5. Generation Hallucination / Incorrect Interpretation

The generator sometimes produces information that does not directly answer the question or is not sufficiently supported by the retrieved context.

34. Future Scope

The project has significant scope for improvement.

1. Improve Sanskrit Text Extraction

A better OCR or layout-aware PDF extraction method can be introduced to reduce broken Sanskrit characters and formatting artifacts.

2. Sanskrit-Specific Preprocessing

Future versions can implement more advanced Sanskrit preprocessing such as:

- Sentence segmentation
- Devanagari normalization
- Better punctuation handling
- Sanskrit-specific linguistic processing

3. Better Embedding Models

The current multilingual MiniLM model provides a good lightweight baseline.

Future versions can compare:

- Larger multilingual embedding models
- Sanskrit-focused embedding models
- Domain-specific embeddings

The models can be compared using Recall@K and semantic retrieval metrics.

4. Hybrid Search

The system can combine:

Semantic Search + Keyword Search

For example:

FAISS Vector Search

+

BM25 Keyword Search

↓

Combined Results

This could improve retrieval when exact Sanskrit words are important.

5. Reranking

A reranker can be placed after FAISS.

Current:

Query → FAISS → Top 3 → LLM

Future:

Query → FAISS → Top 10

↓

Reranker

↓

Top 3

↓

LLM

This could improve the relevance of the context given to the generator.

6. Stronger Generation Model

A stronger multilingual or Sanskrit-capable model could improve:

- Question understanding
- Sanskrit responses
- Translation
- Explanation
- Reasoning
- Faithfulness

Because the assignment requires CPU usage, a **quantized model** would be a practical future direction.

7. Model Quantization

Quantization can reduce model memory and potentially improve CPU inference speed.

For example:

Full Precision Model



Quantized Model



Lower Memory + Faster Inference

This would directly address the current latency problem.

8. Better Prompt Engineering

The prompt can be improved to explicitly instruct the model:

Answer only using the provided context.

Do not add information from outside knowledge.

If the answer is not available, say that it is not present.

Answer in the same language as the question.

This can improve faithfulness.

9. Language-Controlled Output

The application can provide options such as:

Answer Language:

[Sanskrit]

[English]

[Marathi]

This would make the system more useful for different users.

10. Better Evaluation Dataset

A larger evaluation dataset can be created with:

- 30–100 questions
- Direct factual questions
- Explanation questions
- Translation questions
- Multi-step questions
- Questions with different wording
- Sanskrit queries
- Transliteration queries

This would provide a much more reliable evaluation.

11. RAG Evaluation Framework

Future evaluation can include:

- Context Recall
- Context Precision
- Answer Relevance
- Faithfulness
- Recall@K
- MRR
- Cosine Similarity
- End-to-End Latency

This would provide a more complete picture of RAG performance.

12. Confidence-Based Answers

The system can introduce a retrieval confidence threshold.

For example:

If similarity > threshold:

Generate answer

Else:

"The required information was not found in the document."

This can reduce unsupported answers.

13. Source Highlighting

The application can highlight the exact sentence or paragraph used to answer the question.

This would improve explainability and user trust.

14. Larger Document Collection

The current project works with a small Sanskrit document collection.

The architecture can later be extended to:

- Sanskrit books
 - Research papers
 - Historical texts
 - Religious literature
 - Educational documents
 - Multiple Sanskrit PDFs
-

15. Production Deployment

The current Streamlit application can later be deployed using:

- Docker
- Cloud infrastructure
- Optimized inference servers
- Scalable vector databases

This would allow multiple users to access the system.

35. Overall Evaluation of the Project

The project successfully demonstrates the complete concept of a **CPU-based Sanskrit RAG system**.

The indexing pipeline successfully:

11 Pages



Preprocessing



26 Chunks



384-dimensional Embeddings



FAISS Index

The query pipeline successfully:

Question



Embedding



FAISS Search



Top-3 Results



Qwen Generation



Answer + Sources

The evaluation also helped identify the main bottleneck of the system.

The retrieval stage is lightweight and functional, while the main challenges are:

- Sanskrit generation quality
- Answer faithfulness
- CPU generation latency

This is a valuable result because the evaluation does not only demonstrate that the application works; it also identifies exactly which component should be improved in the next iteration.

36. Final Conclusion

The Sanskrit RAG project demonstrates how Retrieval-Augmented Generation can be applied to a specialized language and document domain.

The system successfully integrates document processing, semantic embeddings, vector search, and local language model generation into a single application.

A major strength of the implementation is that it works in a **CPU-only and locally runnable environment**, making it lightweight and suitable for an internship-level AI/ML project.

The use of a multilingual Sentence Transformer provides semantic representations for Sanskrit and mixed-language queries, while FAISS provides efficient local similarity search. The Qwen 2.5 0.5B model provides a lightweight generation component that can operate without a GPU.

The evaluation shows that the retrieval pipeline is functioning and successfully returning relevant source pages. At the same time, the generation results show that the current small language model has difficulty consistently understanding Sanskrit questions and producing grounded answers.

The observed latency of approximately **36–41 seconds** is high for an interactive system, but it is understandable given the CPU-only environment and local language model inference. The results provide a clear opportunity for optimization through quantization, better prompting, improved retrieval, and a stronger multilingual generation model.

Overall, the project provides a strong foundation for a Sanskrit-specific RAG system and demonstrates practical understanding of:

- NLP
- Embeddings
- Semantic Search
- Vector Databases
- Retrieval-Augmented Generation
- LLMs
- Prompt Engineering
- CPU-based inference
- Model evaluation
- Streamlit application development

The project can be further developed into a more accurate and scalable Sanskrit knowledge assistant by improving document extraction, retrieval, generation, evaluation, and inference efficiency.